

Tutorial: Consumo de API e Tratamento de Erros

1. O que é uma API e para que serve

API significa *Application Programming Interface* (Interface de Programação de Aplicações). É uma forma de diferentes sistemas se comunicarem entre si através de regras bem definidas.

Imagine que você está em um restaurante:

- **Você** = Aplicação cliente (navegador, app)
- **Cozinha** = Servidor com os dados
- **Garçom** = API

Você não vai até a cozinha pegar sua comida. O garçom (API) leva seu pedido, busca na cozinha e traz para você. A API funciona da mesma forma: ela recebe seu pedido de dados, busca no servidor e retorna para sua aplicação.

Por que usar APIs?

- **Reutilização de dados:** Você não precisa ter todos os dados, pode usar dados de outros serviços
- **Especialização:** Cada serviço faz o que faz de melhor (previsão do tempo, mapas, dados financeiros)
- **Atualização em tempo real:** Os dados vêm direto da fonte
- **Economia de recursos:** Não precisa armazenar tudo localmente

Exemplos práticos:

- **Previsão do tempo:** Aplicativos consultam APIs meteorológicas
- **Login com Google/Facebook:** Usar APIs de autenticação
- **Mapas:** Google Maps API
- **Pagamentos:** APIs de bancos e cartões

2. Introdução ao fetch()

O `fetch()` é uma função JavaScript moderna para fazer requisições HTTP. É como fazer um pedido para buscar dados de um servidor.

Sintaxe básica:

```
javascript
```

```
fetch('https://exemplo.com/api/dados')
```

O `fetch()` retorna uma **Promise** (promessa), que é um objeto que representa a eventual conclusão ou falha de uma operação assíncrona.

Características do `fetch()`:

- É **nativo** do JavaScript moderno (não precisa de bibliotecas extras)
 - Trabalha de forma **assíncrona** (não trava o navegador)
 - Retorna uma **Promise**
 - É mais simples que o antigo `XMLHttpRequest`
-

3. Requisições GET

GET é o método HTTP mais comum. Ele **busca** (obtem) dados de um servidor sem modificá-los.

Estrutura básica de uma requisição GET:

```
javascript

fetch('https://api.exemplo.com/usuarios')
  .then(response => response.json())
  .then(dados => {
    console.log(dados);
  });
```

Passo a passo:

1. **fetch()** faz a requisição
2. **response.json()** converte a resposta para JSON
3. **dados** contém as informações já prontas para usar

Exemplo prático:

```
javascript

// Buscar dados de um usuário específico
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then(response => response.json())
  .then(usuario => {
    console.log('Nome:', usuario.name);
    console.log('Email:', usuario.email);
  });
```



Exercício 1: Requisições GET

Objetivo: Fazer sua primeira requisição a uma API real.

Use a API pública JSONPlaceholder: <https://jsonplaceholder.typicode.com/posts/1>

Tarefa:

1. Crie um arquivo HTML com JavaScript
2. Faça uma requisição GET para buscar o post de ID 1
3. Exiba no console o título e o corpo do post

Dica: A resposta terá as propriedades `title` e `body`.

<details> <summary>💡 Ver solução</summary> ``html <!DOCTYPE html> <html lang="pt-BR"> <head>
<meta charset="UTF-8"> <title>Exercício 1</title> </head> <body> <h1>Minha primeira requisição</h1>

```
<script>
  fetch('https://jsonplaceholder.typicode.com/posts/1')
    .then(response => response.json())
    .then(post => {
      console.log('Título:', post.title);
      console.log('Corpo:', post.body);
    });
</script>
```

</body> </html> `` </details>

4. Promises e uso de then/catch

O que são Promises?

Uma **Promise** é um objeto que representa o resultado futuro de uma operação assíncrona. Ela pode estar em três estados:

- **Pending** (Pendente): Operação ainda não foi concluída
- **Fulfilled** (Realizada): Operação completada com sucesso
- **Rejected** (Rejeitada): Operação falhou

Usando .then()

O método `.then()` é executado quando a Promise é **resolvida com sucesso**:

```
javascript
```

```
fetch('https://api.exemplo.com/dados')
  .then(response => {
    // Primeiro then: processa a resposta HTTP
    return response.json();
  })
  .then(dados => {
    // Segundo then: trabalha com os dados
    console.log(dados);
  });
```

Usando .catch()

O método `.catch()` captura **erros** que ocorrem em qualquer parte da cadeia:

javascript

```
fetch('https://api.exemplo.com/dados')
  .then(response => response.json())
  .then(dados => console.log(dados))
  .catch(erro => {
    // Captura qualquer erro
    console.error('Erro:', erro);
  });
```

Encadeamento completo:

javascript

```
fetch('https://jsonplaceholder.typicode.com/users')
  .then(response => {
    // Verifica se a resposta foi bem-sucedida
    if (!response.ok) {
      throw new Error('Erro na requisição');
    }
    return response.json();
  })
  .then(usuario => {
    console.log('Usuários recebidos:', usuario);
  })
  .catch(erro => {
    console.error('Algo deu errado:', erro);
  })
  .finally(() => {
    console.log('Requisição finalizada');
  });
```

Exercício 2: Trabalhando com Promises

Objetivo: Praticar o encadeamento de `.then()` e `.catch()`.

Use a API: `https://jsonplaceholder.typicode.com/users`

Tarefas:

1. Faça uma requisição para buscar todos os usuários
2. Filtre apenas os usuários cujo nome começa com "C"
3. Exiba no console o nome e email desses usuários
4. Adicione tratamento de erro com `.catch()`

<details> <summary> 💡 Ver solução</summary> ```javascript

```
fetch('https://jsonplaceholder.typicode.com/users') .then(response => { if (!response.ok) { throw new Error('Erro ao buscar usuários'); } return response.json(); }) .then(usuario => { // Filtra usuários cujo nome começa com C
const usuariosFiltrados = usuarios.filter(user => user.name.startsWith('C') );
```

```
// Exibe informações
usuariosFiltrados.forEach(user => {
  console.log(`Nome: ${user.name}`);
  console.log(`Email: ${user.email}`);
  console.log('---');
});
```

```
})
.catch(erro => {
  console.error('Erro capturado:', erro.message);
});
```

</details>

5. Exibição de dados dinâmicos no HTML

Agora vamos pegar os dados da API e exibir na página HTML de forma dinâmica.

Manipulando o DOM:

```
``javascript
```

```
// Buscar elemento HTML
```

```
const container = document.getElementById('container');
```

```
// Modificar conteúdo
```

```
container.innerHTML = '<p>Novo conteúdo</p>';
```

```
// Adicionar conteúdo
```

```
container.innerHTML += '<p>Mais conteúdo</p>';
```

Exemplo completo:

html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Exibindo dados da API</title>
  <style>
    .usuario {
      border: 1px solid #ddd;
      padding: 15px;
      margin: 10px;
      border-radius: 5px;
    }
  </style>
</head>
<body>
  <h1>Lista de Usuários</h1>
  <div id="usuarios"></div>

  <script>
    const container = document.getElementById('usuarios');

    fetch('https://jsonplaceholder.typicode.com/users')
      .then(response => response.json())
      .then(usuarios => {
        usuarios.forEach(usuario => {
          container.innerHTML += `
            <div class="usuario">
              <h3>${usuario.name}</h3>
              <p><strong>Email:</strong> ${usuario.email}</p>
              <p><strong>Cidade:</strong> ${usuario.address.city}</p>
            </div>
          `;
        });
      })
      .catch(erro => {
        container.innerHTML = '<p style="color: red;">Erro ao carregar usuários</p>';
      });
  </script>
</body>
</html>
```

Boas práticas:

- **Loading:** Mostre um indicador enquanto carrega
- **Placeholder:** Exiba algo enquanto não há dados

- **Template literals:** Use `\${variavel}` para inserir dados
- **Sanitização:** Cuidado com dados que podem conter HTML



Exercício 3: Exibindo dados na página

Objetivo: Criar uma galeria de fotos usando dados de API.

Use a API: `https://jsonplaceholder.typicode.com/photos` (limite aos 10 primeiros)

Tarefas:

1. Crie uma estrutura HTML com um container para as fotos
2. Busque apenas os 10 primeiros registros da API
3. Para cada foto, crie um card mostrando:
 - A imagem (use a propriedade `thumbnailUrl`)
 - O título da foto
4. Estilize os cards com CSS básico

Dica: Para limitar, use: `https://jsonplaceholder.typicode.com/photos?_limit=10`

<details> <summary> 💡 Ver solução</summary> ``html <!DOCTYPE html> <html lang="pt-BR"> <head>
<meta charset="UTF-8"> <title>Galeria de Fotos</title> <style> body { font-family: Arial, sans-serif; padding:
20px; background-color: #f5f5f5; }


```
#galeria {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 20px;
  margin-top: 20px;
}

.foto-card {
  background: white;
  border-radius: 8px;
  padding: 15px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  text-align: center;
}

.foto-card img {
  width: 100%;
  border-radius: 5px;
}

.foto-card h3 {
  font-size: 14px;
  margin-top: 10px;
  color: #333;
}

#loading {
  text-align: center;
  padding: 20px;
  font-size: 18px;
  color: #666;
}

</style>
```

```
</head> <body> <h1>Galeria de Fotos</h1> <div id="loading">Carregando fotos...</div> <div id="galeria">
</div>
```

```
<script>
  const galeria = document.getElementById('galeria');
  const loading = document.getElementById('loading');

  fetch('https://jsonplaceholder.typicode.com/photos?_limit=10')
    .then(response => response.json())
    .then(fotos => {
      loading.style.display = 'none';

      fotos.forEach(foto => {
        galeria.innerHTML += `
          <div class="foto-card">
            
            <h3>${foto.title}</h3>
          </div>
        `;
      });
    })
    .catch(erro => {
      loading.innerHTML = '<p style="color: red;">Erro ao carregar fotos</p>';
      console.error('Erro:', erro);
    });
</script>
```

```
</body> </html> ``` </details>
```

6. Tratamento de erros de rede e resposta

Erros podem acontecer por vários motivos. Vamos aprender a lidar com eles profissionalmente.

Tipos de erros:

1. **Erros de rede:** Internet caiu, servidor fora do ar
2. **Erros de resposta:** Status 404, 500, etc.
3. **Erros de dados:** JSON inválido, formato inesperado

Verificando status da resposta:

```
javascript
```

```
fetch('https://api.exemplo.com/dados')
  .then(response => {
    // Verifica se a resposta foi bem-sucedida (status 200-299)
    if (!response.ok) {
      throw new Error(`Erro HTTP: ${response.status}`);
    }
    return response.json();
  })
  .then(dados => {
    console.log('Sucesso:', dados);
  })
  .catch(erro => {
    console.error('Erro:', erro.message);
  });
```

Tratamento completo:

javascript

```

function buscarDados(url) {
  const statusDiv = document.getElementById('status');

  // Mostra loading
  statusDiv.innerHTML = '<p>Carregando...</p>';

  fetch(url)
    .then(response => {
      // Verifica status HTTP
      if (response.status === 404) {
        throw new Error('Dados não encontrados (404)');
      }
      if (response.status === 500) {
        throw new Error('Erro no servidor (500)');
      }
      if (!response.ok) {
        throw new Error(`Erro: ${response.status}`);
      }

      return response.json();
    })
    .then(dados => {
      statusDiv.innerHTML = '<p style="color: green;">Dados carregados!</p>';
      // Processar dados
    })
    .catch(erro => {
      // Trata diferentes tipos de erro
      if (erro.message.includes('Failed to fetch')) {
        statusDiv.innerHTML = '<p style="color: red;">Erro de conexão. Verifique sua internet.</p>';
      } else {
        statusDiv.innerHTML = `<p style="color: red;">${erro.message}</p>`;
      }
    })
    .finally(() => {
      console.log('Requisição finalizada');
    });
}

```

Feedback visual de erros:

html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Tratamento de Erros</title>
  <style>
    .status {
      padding: 10px;
      margin: 10px 0;
      border-radius: 5px;
    }
    .loading { background: #ffeaa7; }
    .success { background: #55efc4; }
    .error { background: #ff7675; color: white; }
  </style>
</head>
<body>
  <h1>Teste de Tratamento de Erros</h1>

  <button onclick="testarAPI('https://jsonplaceholder.typicode.com/users')">
    Requisição Válida
  </button>

  <button onclick="testarAPI('https://jsonplaceholder.typicode.com/invalid')">
    Requisição com Erro 404
  </button>

  <button onclick="testarAPI('https://url-invalida-que-nao-existe.com/dados')">
    Erro de Conexão
  </button>

  <div id="resultado"></div>

  <script>
    function testarAPI(url) {
      const resultado = document.getElementById('resultado');

      resultado.innerHTML = '<div class="status loading">Carregando...</div>';

      fetch(url)
        .then(response => {
          if (!response.ok) {
            throw new Error(`Erro HTTP: ${response.status}`);
          }
          return response.json();
        })
    }
  </script>
</body>
</html>
```

```

.then(dados => {
  resultado.innerHTML = `
    <div class="status success">
      ✓ Sucesso! Recebidos ${Array.isArray(dados) ? dados.length : 1} registro(s)
    </div>
  `;
})
.catch(erro => {
  let mensagem = "";

  if (erro.message.includes('Failed to fetch')) {
    mensagem = '✗ Erro de rede: Não foi possível conectar ao servidor';
  } else if (erro.message.includes('404')) {
    mensagem = '✗ Erro 404: Recurso não encontrado';
  } else {
    mensagem = `✗ Erro: ${erro.message}`;
  }

  resultado.innerHTML = `<div class="status error">${mensagem}</div>`;
});
}
</script>
</body>
</html>

```

Exercício 4: Tratamento robusto de erros

Objetivo: Criar uma aplicação que busca informações de posts e trata todos os possíveis erros.

Tarefas:

1. Crie um campo de entrada onde o usuário digita um ID de post (1-100)
2. Adicione um botão "Buscar Post"
3. Busque o post na API: `https://jsonplaceholder.typicode.com/posts/[ID]`
4. Trate os seguintes casos:
 - ID não numérico ou vazio
 - Post não encontrado (404)
 - Erro de rede
 - Sucesso (exiba título e corpo do post)
5. Mostre feedback visual apropriado para cada situação

<details> <summary>  Ver solução</summary> ``html <!DOCTYPE html> <html lang="pt-BR"> <head>
<meta charset="UTF-8"> <title>Exercício 4 - Tratamento de Erros</title> <style> body { font-family: Arial,

sans-serif; max-width: 600px; margin: 50px auto; padding: 20px; }

```
.busca {  
  display: flex;  
  gap: 10px;  
  margin: 20px 0;  
}
```

```
input {  
  flex: 1;  
  padding: 10px;  
  font-size: 16px;  
  border: 2px solid #ddd;  
  border-radius: 5px;  
}
```

```
button {  
  padding: 10px 20px;  
  font-size: 16px;  
  background: #0984e3;  
  color: white;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
}
```

```
button:hover {  
  background: #0770c7;  
}
```

```
.mensagem {  
  padding: 15px;  
  margin: 20px 0;  
  border-radius: 5px;  
  display: none;  
}
```

```
.loading {  
  background: #ffea7;  
  display: block;  
}
```

```
.success {  
  background: #55efc4;  
  display: block;  
}
```

```
.error {
```



```
background: #ff7675;
color: white;
display: block;
}

.post {
background: white;
padding: 20px;
border: 2px solid #ddd;
border-radius: 5px;
margin-top: 20px;
}

.post h2 {
margin-top: 0;
color: #2d3436;
}
</style>
```

```
</head> <body> <h1>Buscar Post por ID</h1>
```

```
<div class="busca">
  <input
    type="text"
    id="postId"
    placeholder="Digite o ID do post (1-100)"
    maxlength="3"
  >
  <button onclick="buscarPost()">Buscar Post</button>
</div>
```

```
<div id="mensagem" class="mensagem"></div>
<div id="resultado"></div>
```

```
<script>
function buscarPost() {
  const postId = document.getElementById('postId').value;
  const mensagem = document.getElementById('mensagem');
  const resultado = document.getElementById('resultado');

  // Limpa resultado anterior
  resultado.innerHTML = "";
  mensagem.className = 'mensagem';

  // Validação de entrada
  if (postId.trim() === "") {
    mensagem.className = 'mensagem error';
    mensagem.textContent = '⚠ Por favor, digite um ID';
    return;
  }

  if (isNaN(postId) || postId < 1 || postId > 100) {
    mensagem.className = 'mensagem error';
    mensagem.textContent = '⚠ Digite um número válido entre 1 e 100';
    return;
  }

  // Mostra loading
  mensagem.className = 'mensagem loading';
  mensagem.textContent = '⌚ Buscando post...';

  // Faz a requisição
  fetch(`https://jsonplaceholder.typicode.com/posts/${postId}`)
    .then(response => {
      if (response.status === 404) {
        throw new Error('Post não encontrado');
      }
    })
}
```

```

        if (!response.ok) {
            throw new Error(`Erro ${response.status}: ${response.statusText}`);
        }
        return response.json();
    })
    .then(post => {
        mensagem.className = 'mensagem success';
        mensagem.textContent = '✓ Post encontrado com sucesso!';

        resultado.innerHTML = `
            <div class="post">
                <h2>${post.title}</h2>
                <p><strong>ID:</strong> ${post.id}</p>
                <p><strong>User ID:</strong> ${post.userId}</p>
                <p>${post.body}</p>
            </div>
        `;
    })
    .catch(erro => {
        mensagem.className = 'mensagem error';

        if (erro.message.includes('Failed to fetch')) {
            mensagem.textContent = '✗ Erro de conexão: Verifique sua internet';
        } else if (erro.message.includes('Post não encontrado')) {
            mensagem.textContent = '✗ Post não encontrado. Tente outro ID';
        } else {
            mensagem.textContent = `✗ ${erro.message}`;
        }

        console.error('Erro detalhado:', erro);
    });
}

// Permite buscar ao pressionar Enter
document.getElementById('postId').addEventListener('keypress', function(e) {
    if (e.key === 'Enter') {
        buscarPost();
    }
});
</script>

```

</body> </html> ``` </details>

Mini Projeto Prático: App de Consulta de CEP

Vamos criar uma aplicação completa que consulta endereços através do CEP usando a API pública **ViaCEP**.

Funcionalidades:

- Buscar endereço por CEP
- Validação de entrada
- Loading durante a busca
- Tratamento de erros
- Exibição formatada dos dados

Código completo:

```
html
```

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Consulta de CEP</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      padding: 20px;
    }

    .container {
      background: white;
      padding: 40px;
      border-radius: 20px;
      box-shadow: 0 20px 60px rgba(0,0,0,0.3);
      max-width: 500px;
      width: 100%;
    }

    h1 {
      color: #667eea;
      text-align: center;
      margin-bottom: 30px;
      font-size: 28px;
    }

    .busca-container {
      margin-bottom: 20px;
    }

    label {
      display: block;
      margin-bottom: 8px;
```

```
    color: #333;
    font-weight: 600;
}

.input-group {
    display: flex;
    gap: 10px;
}

input {
    flex: 1;
    padding: 12px;
    font-size: 16px;
    border: 2px solid #ddd;
    border-radius: 8px;
    transition: border-color 0.3s;
}

input:focus {
    outline: none;
    border-color: #667eea;
}

button {
    padding: 12px 24px;
    font-size: 16px;
    background: #667eea;
    color: white;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    transition: background 0.3s;
    font-weight: 600;
}

button:hover {
    background: #5568d3;
}

button:active {
    transform: scale(0.98);
}

.mensagem {
    padding: 12px;
    margin: 20px 0;
    border-radius: 8px;
```

```
    display: none;
    animation: fadeIn 0.3s;
}

@keyframes fadeIn {
    from { opacity: 0; transform: translateY(-10px); }
    to { opacity: 1; transform: translateY(0); }
}

.loading {
    background: #ffeaa7;
    color: #2d3436;
    display: block;
    text-align: center;
}

.success {
    background: #55efc4;
    color: #00b894;
    display: block;
}

.error {
    background: #ff7675;
    color: white;
    display: block;
}

.resultado {
    background: #f8f9fa;
    padding: 20px;
    border-radius: 8px;
    margin-top: 20px;
    display: none;
    animation: fadeIn 0.5s;
}

.resultado.show {
    display: block;
}

.info-item {
    margin-bottom: 15px;
    padding-bottom: 15px;
    border-bottom: 1px solid #ddd;
}
```

```
.info-item:last-child {
  border-bottom: none;
  margin-bottom: 0;
  padding-bottom: 0;
}

.info-label {
  font-size: 12px;
  color: #666;
  text-transform: uppercase;
  font-weight: 600;
  margin-bottom: 5px;
}

.info-value {
  font-size: 16px;
  color: #2d3436;
  font-weight: 500;
}

.dica {
  background: #e3f2fd;
  padding: 15px;
  border-radius: 8px;
  margin-top: 20px;
  font-size: 14px;
  color: #1976d2;
}
</style>
</head>
<body>
  <div class="container">
    <h1>📮 Consulta de CEP</h1>

    <div class="busca-container">
      <label for="cep">Digite o CEP:</label>
      <div class="input-group">
        <input
          type="text"
          id="cep"
          placeholder="00000-000"
          maxlength="9"
        >
        <button onclick="buscarCEP()">Buscar</button>
      </div>
    </div>
  </div>
```



```
<div id="mensagem" class="mensagem"></div>
```

```
<div id="resultado" class="resultado">
```

```
  <div class="info-item">
```

```
    <div class="info-label">CEP</div>
```

```
    <div class="info-value" id="cep-result"></div>
```

```
  </div>
```

```
  <div class="info-item">
```

```
    <div class="info-label">Logradouro</div>
```

```
    <div class="info-value" id="logradouro"></div>
```

```
  </div>
```

```
  <div class="info-item">
```

```
    <div class="info-label">Bairro</div>
```

```
    <div class="info-value" id="bairro"></div>
```

```
  </div>
```

```
  <div class="info-item">
```

```
    <div class="info-label">Cidade</div>
```

```
    <div class="info-value" id="cidade"></div>
```

```
  </div>
```

```
  <div class="info-item">
```

```
    <div class="info-label">Estado</div>
```

```
    <div class="info-value" id="estado"></div>
```

```
  </div>
```

```
</div>
```

```
<div class="dica">
```



Dica: Experimente com CEPs reais, como 01310-100 (Av. Paulista, SP)

```
</div>
```

```
</div>
```

```
<script>
```

```
  // Formata o CEP enquanto digita
```

```
  document.getElementById('cep').addEventListener('input', function(e) {
```

```
    let value = e.target.value.replace(/\D/g, "");
```

```
    if (value.length > 5) {
```

```
      value = value.replace(/^(\d{5})(\d)/, '$1-$2');
```

```
    }
```

```
    e.target.value = value;
```

```
  });
```

```
  // Permite buscar ao pressionar Enter
```

```
  document.getElementById('cep').addEventListener('keypress', function(e) {
```

```
    if (e.key === 'Enter') {
```

```
      buscarCEP();
```

```
    }
```

```
  });
```

```
function buscarCEP() {  
  const cepInput = document.getElementById('cep');  
  const mensagem = document.getElementById('mensagem');  
  const resultado = document.getElementById('resultado');  
  const cep = cepInput.value.replace(/\D/g, "");  
  
  // Esconde resultado anterior  
  resultado.classList.remove('show');  
  mensagem.className = 'mensagem';  
  
  // Validações  
  if (cep === "") {  
    mensagem.className = 'mensagem error';  
    mensagem.textContent = '⚠ Por favor, digite um CEP';  
    return;  
  }  
  
  if (cep.length !== 8) {  
    mensagem.className = 'mensagem error';  
    mensagem.textContent = '⚠ CEP deve ter 8 dígitos';  
    return;  
  }  
  
  // Mostra loading  
  mensagem.className = 'mensagem loading';  
  mensagem.textContent = '⌚ Buscando informações...';  
  
  // Faz a requisição  
  fetch(`https://viacep.com.br/ws/${cep}/json/`)  
    .then(response => {  
      if (!response.ok) {  
        throw new Error('Erro na requisição');  
      }  
      return response.json();  
    })  
    .then(dados => {  
      // Verifica se o CEP foi encontrado  
      if (dados.erro) {  
        throw new Error('CEP não encontrado');  
      }  
  
      // Mostra sucesso  
      mensagem.className = 'mensagem success';  
      mensagem.textContent = '✓ CEP encontrado com sucesso!';  
  
      // Preenche os dados  
      document.getElementById('cep-result').textContent = dados.cep;
```

```

document.getElementById('logradouro').textContent = dados.logradouro || 'Não informado';
document.getElementById('bairro').textContent = dados.bairro || 'Não informado';
document.getElementById('cidade').textContent = dados.localidade;
document.getElementById('estado').textContent = `${dados.uf} - ${dados.estado}`;

// Mostra resultado
resultado.classList.add('show');
}))
.catch(erro => {
  mensagem.className = 'mensagem error';

  if (erro.message === 'CEP não encontrado') {
    mensagem.textContent = '✗ CEP não encontrado. Verifique e tente novamente';
  } else if (erro.message.includes('Failed to fetch')) {
    mensagem.textContent = '✗ Erro de conexão. Verifique sua internet';
  } else {
    mensagem.textContent = '✗ Erro ao buscar CEP. Tente novamente';
  }

  console.error('Erro:', erro);
});
}
</script>
</body>
</html>

```

Como testar:

1. Abra o arquivo HTML no navegador
2. Digite um CEP válido (ex: 01310-100)
3. Clique em "Buscar" ou pressione Enter
4. Veja as informações aparecerem

Desafios extras:

- Adicione um histórico de CEPs consultados
- Crie um botão para limpar o formulário
- Adicione mais validações
- Salve os últimos CEPs no localStorage

Criando sua própria API com Python Flask

Agora vamos criar uma API simples do zero usando Python e Flask!

Passo 1: Instalar o Flask

```
bash
```

```
pip install flask flask-cors
```

Passo 2: Criar a API (api.py)

```
python
```

```
from flask import Flask, jsonify, request
```

```
from flask_cors import CORS
```

```
app = Flask(__name__)
```

```
CORS(app) # Permite requisições de outros domínios
```

```
# Dados de exemplo (banco de dados simulado)
```

```
livros = [
```

```
    {"id": 1, "titulo": "1984", "autor": "George Orwell", "ano": 1949},
```

```
    {"id": 2, "titulo": "Dom Casmurro", "autor": "Machado de Assis", "ano": 1899},
```

```
    {"id": 3, "titulo": "O Hobbit", "autor": "J.R.R. Tolkien", "ano": 1937},
```

```
    {"id": 4, "titulo": "Cem Anos de Solidão", "autor": "Gabriel García Márquez", "ano": 1967},
```

```
]
```

```
# Rota principal
```

```
@app.route('/')
```

```
def home():
```

```
    return jsonify({
```

```
        "mensagem": "Bem-vindo à API de Livros!",
```

```
        "rotas": {
```

```
            "/livros": "Lista todos os livros",
```

```
            "/livros/<id>": "Busca um livro por ID",
```

```
            "/livros/buscar?autor=nome": "Busca livros por autor"
```

```
        }
```

```
    })
```

```
# GET - Listar todos os livros
```

```
@app.route('/livros', methods=['GET'])
```

```
def listar_livros():
```

```
    return jsonify(livros)
```

```
# GET - Buscar livro por ID
```

```
@app.route('/livros/<int:id>', methods=['GET'])
```

```
def buscar_livro(id):
```

```
    livro = next((l for l in livros if l['id'] == id), None)
```

```
    if livro:
```

```
        return jsonify(livro)
```

```
    else:
```

```
        return jsonify({"erro": "Livro não encontrado"}), 404
```

```
# GET - Buscar livros por autor
```

```
@app.route('/livros/buscar', methods=['GET'])
```

```
def buscar_por_autor():
```

```
    autor = request.args.get('autor', "").lower()
```

```

if not autor:
    return jsonify({"erro": "Parâmetro 'autor' é obrigatório"}), 400

resultados = [l for l in livros if autor in l['autor'].lower()]

return jsonify(resultados)

# POST - Adicionar novo livro
@app.route('/livros', methods=['POST'])
def adicionar_livro():
    dados = request.get_json()

    # Validações
    if not dados or 'titulo' not in dados or 'autor' not in dados:
        return jsonify({"erro": "Título e autor são obrigatórios"}), 400

    # Gera novo ID
    novo_id = max([l['id'] for l in livros]) + 1

    novo_livro = {
        "id": novo_id,
        "titulo": dados['titulo'],
        "autor": dados['autor'],
        "ano": dados.get('ano', None)
    }

    livros.append(novo_livro)

    return jsonify(novo_livro), 201

if __name__ == '__main__':
    print("🚀 API iniciada em http://localhost:5000")
    app.run(debug=True, port=5000)

```

Passo 3: Executar a API

```

bash

python api.py

```

A API estará rodando em `http://localhost:5000`

Passo 4: Criar o Frontend (index.html)

```

html

```

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Biblioteca - Cliente API</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
      padding: 20px;
    }

    .container {
      max-width: 1000px;
      margin: 0 auto;
      background: white;
      padding: 40px;
      border-radius: 20px;
      box-shadow: 0 20px 60px rgba(0,0,0,0.3);
    }

    h1 {
      color: #667eea;
      text-align: center;
      margin-bottom: 30px;
    }

    .acoes {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
      gap: 15px;
      margin-bottom: 30px;
    }

    button {
      padding: 12px 20px;
      font-size: 14px;
      background: #667eea;
```

```
    color: white;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    transition: all 0.3s;
    font-weight: 600;
}

button:hover {
    background: #5568d3;
    transform: translateY(-2px);
}

.busca {
    display: flex;
    gap: 10px;
    margin-bottom: 30px;
}

input {
    flex: 1;
    padding: 12px;
    font-size: 16px;
    border: 2px solid #ddd;
    border-radius: 8px;
}

input:focus {
    outline: none;
    border-color: #667eea;
}

#status {
    padding: 15px;
    margin-bottom: 20px;
    border-radius: 8px;
    display: none;
}

#status.show {
    display: block;
}

#status.loading {
    background: #ffeaa7;
    color: #2d3436;
}
```



```
#status.success {
  background: #55efc4;
  color: #00b894;
}

#status.error {
  background: #ff7675;
  color: white;
}

.livros-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 20px;
}

.livro-card {
  background: #f8f9fa;
  padding: 20px;
  border-radius: 8px;
  border-left: 4px solid #667eea;
  transition: transform 0.3s;
}

.livro-card:hover {
  transform: translateY(-5px);
  box-shadow: 0 5px 15px rgba(0,0,0,0.1);
}

.livro-card h3 {
  color: #2d3436;
  margin-bottom: 10px;
}

.livro-card p {
  color: #636e72;
  margin-bottom: 5px;
}

.livro-card .id {
  font-size: 12px;
  color: #b2bec3;
}

.formulario {
  background: #f8f9fa;
```

```

padding: 20px;
border-radius: 8px;
margin-bottom: 30px;
display: none;
}

.formulario.show {
display: block;
}

.form-group {
margin-bottom: 15px;
}

.form-group label {
display: block;
margin-bottom: 5px;
color: #2d3436;
font-weight: 600;
}

.form-group input {
width: 100%;
}

.form-actions {
display: flex;
gap: 10px;
}
</style>
</head>
<body>
<div class="container">
<h1><img alt="book icon" data-bbox="138 693 160 708" style="vertical-align: middle;"/> Biblioteca Virtual</h1>

<div class="acoes">
<button onclick="listarTodos()"><img alt="book icon" data-bbox="355 755 377 770" style="vertical-align: middle;"/> Listar Todos</button>
<button onclick="mostrarBusca()"><img alt="magnifying glass icon" data-bbox="375 775 397 790" style="vertical-align: middle;"/> Buscar por Autor</button>
<button onclick="mostrarFormulario()"><img alt="plus icon" data-bbox="410 795 432 810" style="vertical-align: middle;"/> Adicionar Livro</button>
</div>

<div class="busca" id="buscaAutor" style="display: none;">
<input
type="text"
id="autorInput"
placeholder="Digite o nome do autor"
>

```

```
<button onclick="buscarPorAutor()">Buscar</button>
</div>

<div class="formulario" id="formularioLivro">
  <h2>Adicionar Novo Livro</h2>
  <div class="form-group">
    <label for="tituloInput">Título*</label>
    <input type="text" id="tituloInput" placeholder="Digite o título">
  </div>
  <div class="form-group">
    <label for="autorFormInput">Autor*</label>
    <input type="text" id="autorFormInput" placeholder="Digite o autor">
  </div>
  <div class="form-group">
    <label for="anoInput">Ano</label>
    <input type="number" id="anoInput" placeholder="Digite o ano (opcional)">
  </div>
  <div class="form-actions">
    <button onclick="adicionarLivro()">Salvar</button>
    <button onclick="esconderFormulario()" style="background: #dfe6e9; color: #2d3436;">Cancelar</button>
  </div>
</div>

<div id="status"></div>

<div id="resultado" class="livros-grid"></div>
</div>

<script>
  const API_URL = 'http://localhost:5000';

  function mostrarStatus(mensagem, tipo) {
    const status = document.getElementById('status');
    status.textContent = mensagem;
    status.className = `show ${tipo}`;

    if (tipo === 'success' || tipo === 'error') {
      setTimeout(() => {
        status.classList.remove('show');
      }, 3000);
    }
  }

  function mostrarLivros(livros) {
    const resultado = document.getElementById('resultado');

    if (livros.length === 0) {
```

```
    resultado.innerHTML = '<p style="text-align: center; color: #636e72;">Nenhum livro encontrado</p>';  
    return;  
}
```

```
resultado.innerHTML = livros.map(livro => `  
    <div class="livro-card">  
        <div class="id">ID: ${livro.id}</div>  
        <h3>${livro.titulo}</h3>  
        <p><strong>Autor:</strong> ${livro.autor}</p>  
        <p><strong>Ano:</strong> ${livro.ano || 'Não informado'}</p>  
    </div>  
`).join("");  
}
```

```
function listarTodos() {  
    mostrarStatus('⌚ Carregando livros...', 'loading');  
  
    fetch(`${API_URL}/livros`)  
        .then(response => {  
            if (!response.ok) {  
                throw new Error('Erro ao buscar livros');  
            }  
            return response.json();  
        })  
        .then(livros => {  
            mostrarStatus('✅ ${livros.length} livro(s) encontrado(s)', 'success');  
            mostrarLivros(livros);  
        })  
        .catch(erro => {  
            mostrarStatus('❌ Erro: ' + erro.message, 'error');  
            console.error('Erro:', erro);  
        });  
}
```

```
function mostrarBusca() {  
    const busca = document.getElementById('buscaAutor');  
    busca.style.display = busca.style.display === 'none' ? 'flex' : 'none';  
    esconderFormulario();  
}
```

```
function buscarPorAutor() {  
    const autor = document.getElementById('autorInput').value.trim();  
  
    if (!autor) {  
        mostrarStatus('⚠ Digite o nome do autor', 'error');  
        return;  
    }  
}
```

```
mostrarStatus('⌚ Buscando...', 'loading');
```

```
fetch(`${API_URL}/livros/buscar?autor=${encodeURIComponent(autor)}`)  
  .then(response => response.json())  
  .then(livros => {  
    if (livros.length === 0) {  
      mostrarStatus('Nenhum livro encontrado para este autor', 'error');  
    } else {  
      mostrarStatus(`✓ ${livros.length} livro(s) encontrado(s)`, 'success');  
    }  
    mostrarLivros(livros);  
  })  
  .catch(erro => {  
    mostrarStatus('✗ Erro na busca', 'error');  
    console.error('Erro:', erro);  
  });  
}
```

```
function mostrarFormulario() {  
  const form = document.getElementById('formularioLivro');  
  form.classList.toggle('show');  
  document.getElementById('buscaAutor').style.display = 'none';  
}
```

```
function esconderFormulario() {  
  document.getElementById('formularioLivro').classList.remove('show');  
  limparFormulario();  
}
```

```
function limparFormulario() {  
  document.getElementById('tituloInput').value = '';  
  document.getElementById('autorFormInput').value = '';  
  document.getElementById('anoInput').value = '';  
}
```

```
function adicionarLivro() {  
  const titulo = document.getElementById('tituloInput').value.trim();  
  const autor = document.getElementById('autorFormInput').value.trim();  
  const ano = document.getElementById('anoInput').value;  
  
  if (!titulo || !autor) {  
    mostrarStatus('⚠ Título e autor são obrigatórios', 'error');  
    return;  
  }  
  
  mostrarStatus('⌚ Adicionando livro...', 'loading');
```

```

const novoLivro = {
  titulo: titulo,
  autor: autor
};

if (ano) {
  novoLivro.ano = parseInt(ano);
}

fetch(`${API_URL}/livros`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(novoLivro)
})
  .then(response => {
    if (!response.ok) {
      throw new Error('Erro ao adicionar livro');
    }
    return response.json();
  })
  .then(livro => {
    mostrarStatus('✓ Livro adicionado com sucesso!', 'success');
    esconderFormulario();
    listarTodos();
  })
  .catch(erro => {
    mostrarStatus('✗ Erro ao adicionar: ' + erro.message, 'error');
    console.error('Erro:', erro);
  });
}

// Carrega os livros ao iniciar
listarTodos();
</script>
</body>
</html>

```

Como usar:

1. Execute a API Python:

```
bash
```

2. Abra o HTML no navegador:

- Simplesmente abra o arquivo `index.html`

3. Teste as funcionalidades:

- Listar todos os livros
- Buscar por autor
- Adicionar novos livros

Endpoints da API:

- `GET /` - Informações da API
- `GET /livros` - Lista todos os livros
- `GET /livros/<id>` - Busca livro por ID
- `GET /livros/buscar?autor=nome` - Busca por autor
- `POST /livros` - Adiciona novo livro

Conclusão

Parabéns! Você aprendeu:

- ✓ O que são APIs e como funcionam
- ✓ Como usar `fetch()` para fazer requisições
- ✓ Trabalhar com Promises (`.then()` e `.catch()`)
- ✓ Exibir dados dinâmicos no HTML
- ✓ Tratar erros de forma profissional
- ✓ Criar sua própria API com Flask
- ✓ Integrar frontend e backend

Próximos passos:

1. **Aprenda `async/await`:** Forma moderna de trabalhar com Promises
2. **Explore outras APIs:** [Public APIs](#)
3. **Aprenda sobre autenticação:** Tokens, JWT, OAuth
4. **Estude banco de dados:** SQLite, PostgreSQL, MongoDB
5. **Pratique mais:** Crie projetos pessoais

Recursos úteis:

- [MDN - Fetch API](#)

- [JSONPlaceholder](#) - API para testes
- [ViaCEP](#) - API de CEPs brasileiros
- [Flask Documentation](#)

Continue praticando e bons estudos! 🚀